Lecture 9. Pipes

Alternatives to the pipe, when not to use it, other magrittr tools

Contents

9.1 Why pipes **2**

9.2 When not to use the pipe **2**

9.3 Other tools from magrittr **3**

9.4 Selected exercises **3**

9.1 Why pipes

Motivation. Code communicates; the pipe reads in the order of the analysis.

`%>%` is from `magrittr` (loaded by the tidyverse). Four ways to write "Little bunny Foo Foo went hopping through the forest, scooping up the field mice and bopping them on the head":

Save each intermediate step.

```
foo_foo <- little_bunny()
foo_foo_1 <- hop(foo_foo, through = forest)
foo_foo_2 <- scoop(foo_foo_1, up = field_mice)
foo_foo_3 <- bop(foo_foo_2, on = head)
```

Cluttered names, error-prone suffixes. Memory is not the issue: copies share unchanged columns.

Overwrite the original.

```
foo_foo <- hop(foo_foo, through = forest)
foo_foo <- scoop(foo_foo, up = field_mice)
foo_foo <- bop(foo_foo, on = head)
```

Debugging means rerunning from the start; the repeated name hides what changes.

Function composition.

```
bop(scoop(hop(foo_foo, through = forest), up = field_mice), on = head)
```

Read inside out; arguments far from their function ("Dagwood sandwich").

The pipe.

```
foo_foo %>%
  hop(through = forest) %>%
  scoop(up = field_mice) %>%
  bop(on = head)
```

A sequence of verbs: hops, then scoops, then bops. magrittr rewrites it roughly as

```
my_pipe <- function(.) {
  . <- hop(., through = forest)
  . <- scoop(., up = field_mice)
  bop(., on = head)
}
```

So it fails with functions using the current environment (`assign()`, `get()`, `load()`) or special lazy evaluation (`tryCatch()`, `try()`, `suppressMessages()`).

The native pipe [Supplement]

R 4.1's `|>`: `x |> f(y)` is `f(x, y)`; faster, no package, but no `.` placeholder (R 4.2: `_` for named arguments) and no special pipes. These notes use `%>%`.

9.2 When not to use the pipe

Pipes suit a linear chain on one object. Avoid them when

- there are more than about ten steps (name intermediate objects);
- there are several inputs or outputs;
- the steps form a directed graph, not a chain.

9.3 Other tools from magrittr

Tee pipe `%T>%` returns its left-hand side, so a side-effect step (plot, print, save) does not end the chain; otherwise `plot()` passes `NULL`:

```
rnorm(100) %>%
  matrix(ncol = 2) %>%
  plot() %>%
  str()
```

```
##  NULL
```

```
rnorm(100) %>%
  matrix(ncol = 2) %T>%
  plot() %>%
  str()
```

```
##  num [1:50, 1:2] -0.387 -0.785 -1.057 -0.796 -1.756 ...
```

Exposition pipe `%$%` exposes columns by name to functions without a `data` argument:

```
mtcars %$% cor(disp, mpg)
```

```
## [1] -0.8475514
```

Assignment pipe `%<>%`: `mtcars %<>% transform(cyl = cyl * 2)` for `mtcars <- mtcars %>% transform(cyl = cyl * 2)`; often avoided, as assignment should be visible at the line start.

9.4 Selected exercises

Motivation. Consequences of the pipe's rewriting rule.

Exercise 9.1 (the rewriting rule)

Take as definition that `x %>% f(y)` means `f(x, y)` when the dot `.` does not appear as an argument of `f`, and means `f(y)` with `.` replaced by `x` when it does.
(a) Prove that a chain `x %>% f1(y1) %>% ⋯ %>% fk(yk)` without dots evaluates to $f_k(\cdots f_2(f_1(x, y_1), y_2) \cdots, y_k)$.
(b) Predict, then check, the values of `1:3 %>% c(4, .)` and `1:3 %>% c(length(.))`.

Solution [Supplement].

(a)

Step 1. Left-associative: `((x %>% f1(y1)) %>% f2(y2)) %>% ⋯`. Induct on k; $k = 1$ is the definition.

Step 2. If length $k-1$ gives $v = f_{k-1}(\cdots f_1(x, y_1) \cdots, y_{k-1})$, length k pipes v into `fk(yk)`: $f_k(v, y_k)$.

(b) In `c(4, .)` the dot is an argument: `c(4, 1:3)`. In `c(length(.))` it is only inside `length(.)`, so the left side is still inserted first: `c(1:3, length(1:3))`:

```
1:3 %>% c(4, .)
```

```
## [1] 4 1 2 3
```

```
1:3 %>% c(length(.))
```

```
## [1] 1 2 3 3
```

■

Exercise 9.2 (assignment inside a pipe)

Explain the following output. Why does `assign()` inside a pipe not create a variable unless the environment is given explicitly?

```
"zz" %>% assign(100)
exists("zz")
```

```
## [1] FALSE
```

```
env <- environment()
"zz" %>% assign(100, envir = env)
exists("zz")
```

```
## [1] TRUE
```

Solution [Supplement].

Step 1. `assign(x, value)` writes into the environment where it is evaluated (default `pos = -1`).

Step 2. The pipe evaluates the right side in a temporary function-like environment binding `.` (cf. `my_pipe`); `zz` is created there and vanishes, so `exists("zz")` is `FALSE`.

Step 3. Passing the captured calling environment as `envir` makes `zz` visible. Likewise `get()`, `load()` see the pipe's environment, not ours. ■

Exercise 9.3 (when the exposition pipe is needed)

Why does the first call below fail while the second works?

```
mtcars %>% sum(cyl)
```

```
## Error: object 'cyl' not found
```

```
mtcars %$% sum(cyl)
```

```
## [1] 198
```

Solution [Supplement]. By Exercise 9.1 the first is `sum(mtcars, cyl)`, and `cyl` is looked up in the calling environment, where it does not exist (`sum()` has no `data` argument, unlike `lm()`). `%$%` evaluates `sum(cyl)` with the columns as variables, like `with(mtcars, sum(cyl))`: 198 cylinders. ■

Exercise 9.4 (the native pipe is syntax)

Explain the following outputs, and why `1:3 |> sum` is an error while `1:3 %>% sum` works.

```
quote(x |> f(y))
```

```
## f(x, y)
```

```
quote(x %>% f(y))
```

```
## x %>% f(y)
```

```
tryCatch(str2lang("1:3 |> sum"), error = conditionMessage)
```

```
## [1] "The pipe operator requires a function call as RHS (<input>:1:8)"
```

Solution [Supplement].

Step 1. `quote()` does not evaluate. The parser rewrites `x |> f(y)`: it *is* `f(x, y)`, no pipe function, no overhead.

Step 2. `` `%>%` `` is an ordinary two-argument function: the quoted call stays a call to it, rewritten only when evaluated.

Step 3. The parser inserts `x` into a call, so `|>` needs `sum()`; magrittr gets the unevaluated right side and accepts a bare name: `1:3 %>% sum` is `sum(1:3)`. ■